33. Search in Rotated Sorted Array

PLATFORM	DIFFICULTY	DATE
Platform: LeetCode	MEDIUM	Solved: October 10, 2026

Problem Summary

Given a sorted array rotated at an unknown pivot and a target value, return the index of the target or -1 if not found. Must run in O(log n).

Approaches

- **Brute Force O(n)**: Linear scan through array — find target directly. Too slow.
- **Optimal O(log n)**: Modified binary search — one half is always sorted even in a rotated array. Check which half is sorted, then check if target lies in that half, search accordingly.

Approach

Modified binary search. At every mid, one of the two halves is always sorted. Identify the sorted half using nums[left] <= nums[mid]. Check if target falls in the sorted half — if yes search there, else search the other half.

Algo

1. left = 0, right = nums.length - 1.
2. While left <= right: mid = (left + right) / 2.
3. If nums[mid] == target return mid.
4. If nums[left] <= nums[mid] (left half sorted): if nums[left] <= target < nums[mid] → right = mid-1. Else → left = mid+1.
5. Else (right half sorted): if nums[mid] < target <= nums[right] → left = mid+1. Else → right = mid-1.
6. Return -1.

Key Insights

- In a rotated array, one half is ALWAYS sorted — this enables binary search.
- nums[left] <= nums[mid] identifies sorted left half — use <= to handle edge cases.
- After identifying sorted half, check if target is strictly within that half's range.
- If target not in sorted half → must be in the other half → search there.

Dry Run

left	right	mid	nums[mid]	sorted half	action
0	6	3	7	left(4<=7)	0 not in [4..7] → left=4
4	6	5	1	right(1<=2)	0 not in [1..2] → right=4
4	4	4	0	found!	return 4

nums = [4,5,6,7,0,1,2], target = 0. Answer: index 4 ■

Observations

- nums[left] <= nums[mid] uses <= not < — handles case where left == mid (single element).
- LC 81 (with duplicates): when nums[left] == nums[mid], can't determine sorted half → do left++ and continue.
- No need to find pivot separately — sorted-half check handles all rotation cases implicitly.
- Works regardless of where the pivot is — fully general solution.

Section

Time Complexity

O(log n) — binary search eliminates half the array each step.

Space Complexity

O(1) — only pointers used.

Points to Remember

- Key insight: one half is always sorted in a rotated array — identify using nums[left] <= nums[mid].
- Check target range in sorted half: nums[left] <= target < nums[mid] for left; nums[mid] < target <= nums[right] for right.
- Use <= in boundary comparisons — handles equal values and single-element arrays.
- LC 81 follow-up: when nums[left] == nums[mid], can't determine sorted half → do left++ and continue.
- No need to find pivot separately — sorted-half check handles all rotation cases implicitly.

Solution Code — Java

```
class Solution {
    public int search(int[] nums, int target) {
        int left = 0, right = nums.length - 1;

        while (left <= right) {
            int mid = (left + right) / 2;

            if (nums[mid] == target) return mid;

            if (nums[left] <= nums[mid]) {
                // left half is sorted
                if (nums[left] <= target && target < nums[mid])
                    right = mid - 1;
                else
                    left = mid + 1;
            } else {
                // right half is sorted
                if (nums[mid] < target && target <= nums[right])
                    left = mid + 1;
                else
                    right = mid - 1;
            }
        }
        return -1;
    }
}
```